BAB VII
TRIGGER

A. Tujuan

1. Mahasiswa dapat menjelaskan struktur sintaks dan komponen di dalam pembuatan *trigger*.
2. Mahasiswa dapat mengimplementasikan pembuatan *trigger* dalam berbagai operasi *insert*, *update*, dan *delete*.
3. Mahasiswa dapat mengimplementasikan pemanggilan *stored function* di dalam *trigger*.

B. Uraian Materi

Sebuah basis data dapat memproses permintaan perubahan datanya hingga ribuan data per detiknya. Permintaan perubahan data meliputi perintah-perintah *insert, edit, alter,* atau *delete*. Terkait perubahan data, dibutuhkan manajemen data yang bagus sehingga dapat mempertahankan konsistensi pada data yang saling memiliki hubungan secara logika dengan melakukan validasi sebelum perubahan tersebut disimpan. Sebelum MySql versi 5.0.2, Mysql hanya dapat melakukan validasi data meliputi pembatasan masukan sesuai tipe data pada kolom, penggunaan *foreign key* untuk memastikan *referential integrity*. Sejak versi MySQL 5.0.2, validasi perubahan data yang dilakukan oleh perintah-perintah *insert, edit, alter,* atau *delete* dapat dilakukan menggunakan *trigger*.

Trigger adalah kumpulan statemen yang disimpan dan dijalankan ketika suatu event terjadi pada suatu kolom atau tabel tertentu (Kruckenberg & Pipes 2005). Umumnya *trigger* digunakan untuk menjalankan statemen *query* secara otomatis pada kondisi sebelum atau sesudah proses INSERT, UPDATE atau DELETE pada bagian dari suatu tabel atau kolom. Sehingga pemicu eksekusi statemen di dalam *trigger* adalah perintah INSERT, UPDATE atau DELETE pada tabel atau kolom tertentu (Kruckenberg & Pipes 20025).

Sebagai contoh, misalnya untuk menyimpan id pelanggan secara otomatis ke tabel "log" sebelum proses menghapus data di tabel pelanggan, melakukan *update* data secara otomatis jika terjadi perubahan dalam sistem penjualan misalkan ada proses entri barang baru sehingga stok barang tersebut akan bertambah secara otomatis, atau menyimpan setiap aktivitas pada basis data ke dalam tabel "log". Contoh lain penggunaan *trigger* adalah untuk

memperbaharui nilai *date time* suatu kolom pada tabel dalam *database* perpustakaan. Apabila terdapat transaksi baru peminjaman buku, *trigger* dapat digunakan untuk mengubah tanggal kembali pada rentang 7 (tujuh) hari berdasarkan tanggal peminjaman.

Berdasarkan contoh yang telah disebutkan, maka *trigger* merupakan mekanisme yang baik untuk menjaga integritas data. Selain itu, *trigger* dapat digunakan untuk mengotomatisasi proses di dalam *database* (Harrison & Feuerstein 2006). Melalui fitur *activation time* pada *trigger*, yaitu *before* dan *after*, dapat dilakukan pemeriksaan data sebelum data tersebut ditambahkan (*Insert)* ke dalam suatu tabel (DuBois 2014).

1. Sintak *Trigger*

Komponen yang harus ada di dalam *trigger* adalah 1) nama trigger, 2) *activation time*, 3) kondisi aktifasi *trigger,* 4) tabel yang akan mengaktifkan *trigger* sesuai kondisi *trigger* yang diberikan, dan 5) aksi yang dilakukan oleh *trigger*. *Trigger* dapat dibuat menggunakan perintah "*CREATE TRIGGER*". Sintaks *trigger* secara umum ditunjukkan pada gambar 7.1.

```
CREATE [DEFINER={user|CURRENT_USER}] TRIGGER trigger_name
  {BEFORE|AFTER}
  {UPDATE|INSERT|DELETE}
ON table_name
FOR EACH ROW
trigger_statements
```

Gambar 7.1. Struktur Sintaks *Trigger*

Berdasarkan gambar 7.1. terdapat beberapa komponen penyusun *trigger*. Penjelasan tiap-tiap komponen tersebut adalah sebagai berikut (Harrison & Feuerstein 2006) :

a. *DEFINER={user | CURRENT_USER}*. Digunakan untuk mengendalikan akun yang diberikan hak akses (*privileges*) ketika *trigger* dijalankan. *Current_user* menunjukkan bahwa *trigger* hanya dapat dieksekusi oleh akun pembuat *trigger* tersebut.

b. *trigger_name*. Merupakan nama *trigger*, di dalam satu tabel hanya diperbolehkan ada satu *trigger* untuk tiap *activation time*. Sebagai contoh, hanya diijinkan terdapat satu

trigger untuk *BEFORE INSERT*, atau *AFTER INSERT* pada suatu tabel. Begitu pula dengan perintah DML pada tabel tersebut.

c. *BEFORE|AFTER*. Menentukan pemicu dijalankannya *trigger*, apakah sebelum (*BEFORE*) atau setelah *(AFTER*) perintah DML dijalankan pada suatu tabel.

d. *UPDATE|INSERT|DELETE*. Mendefinisikan perintah DML dimana *trigger* akan ditautkan.

e. *ON table_name*. Menentukan tabel yang ditautkan pada suatu *trigger*.

f. *FOR EACH ROW*. Klausa ini bersifat wajib di dalam *trigger*. Klausa "*FOR EACH ROW*" menunjukkan bahwa *trigger* akan dijalankan satu kali untuk setiap baris data yang dipengaruhi oleh pernyataan DML.

g. *trigger_statements*. Digunakan untuk menentukan perintah di dalam *trigger* yang akan dijalankan. Perintah di dalam *trigger* harus diapit oleh blok "BEGIN"…"END".

Sebagai contoh, dibuat *trigger* untuk menyimpan proses penambahan data yang terjadi pada tabel "mahasiswa" ke dalam tabel "log". Sebelumnya buat terlebih dahulu table dengan nama "tb_log" dengan sintak seperti ditunjukkan pada gambar 7.2.

```
CREATE TABLE `tb_log` (
        `log_id` int(8) NOT NULL AUTO_INCREMENT,
        `Nim` Varchar(12) NOT NULL,
        `perubahan` enum('CREATE','UPDATE','DELETE') NOT NULL,
        `waktu` timestamp NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
        PRIMARY KEY (`log_id`)
)
```

Gambar 7.2. Sintaks Pembuatan Tabel "tb_log"

Setelah tabel "tb_log" dibuat, kemudian dibuat *trigger* untuk menambahkan data secara otomatis ke dalam tabel "tb_log" ketika terjadi penambahan data di table "mahasiswa". *Activation time* yang digunakan adalah *"AFTER INSERT"*. Artinya penambahan data di tabel "tb_log" akan dijalankan setelah proses *insert* pada tabel "mahasiswa" sukses. Sintak trigger ditunjukkan pada gambar 7.3. Setelah *trigger* sukses dibuat, maka dapat di bagian *design* tabel yang ditautkan dengan *trigger* sebagaimana ditunjukkan pada gambar 7.4.

```
DELIMITER $$
CREATE TRIGGER `after_insert_mahasiswa`
    AFTER INSERT ON `mahasiswa` FOR EACH ROW
    BEGIN
                        SET @tipe = 'CREATE';
      INSERT INTO tb_log (Nim, perubahan,waktu) VALUES (NEW.Nim, @tipe,now());
    END;
$$
```

Gambar 7.3. Sintaks *Trigger Insert* Data Mahasiswa

Fields | Indexes | Foreign Keys | Triggers | Options | Comment | SQL Preview

Name	Fires	Insert	Update	Delete
buat_NIM	BEFORE	☑	☐	☐
after_insert_mahasiswa	AFTER	☑	☐	☐
after_update_mahasiswa	AFTER	☐	☑	☐

Gambar 7.4. *Trigger* pada Tabel "mahasiswa"

Hasil *trigger* pada gambar 7.3. dapat diketahui ketika ada penambahan data di tabel "mahasiswa". Sebagai contoh pada tabel "mahasiswa" ditambahkan satu data. Hasil eksekusi *trigger* dapat dilihat pada gambar 7.5.

200304005	Wahyu Andreas	1992-08-06	JL STASIUN NOTOG NO 2 RT 06/02 NOTOG PATIKRAJA 53171	Laki-laki

log_id	perubahan	waktu	Nim
36	CREATE	2021-02-08 17:35:34	200304005

Gambar 7.5. Hasil Eksekusi *Trigger*

Sesuai hasil yang ditunjukkan pada gambar 7.5., setelah data mahasiswa ditambahkan ke tabel "mahasiswa', yaitu mahasiswa dengan NIIM "200304005. Selanjutnya pada tabel "tb_log" akan ditambahkan satu *record* baru dengan isi perubahan adalah "CREATE" dan nilai pada *field* NIM adalah "200304005". *Trigger* dapat mengakses record tabel sebelum atau sesudah proses dengan menggunakan *NEW* dan *OLD*. *NEW* digunakan untuk mengambil record yang akan diproses (*insert atau update*), sedangkan *OLD* digunakan untuk mengakses *record* yang sudah diproses (*update* atau *delete*) (Solichin 2010).

2. *Trigger* untuk Update Data

Selain untuk menangani ketika terjadi proses penambahan data pada suatu tabel, *trigger* juga dapat digunakan untuk menangani proses perubahan data yang terkait dengan data lainnya. Sebagai contoh, ketika terdapat perubahan kode mata kuliah di tabel "mata kuliah", maka kode mata kuliah yang berelasi di tabel "jadwal" juga akan mengalami

perubahan untuk menjaga konsistensi data. Sintak *trigger* untuk menangani perubahan data ditunjukkan pada gambar 7.6.

```
DELIMITER $$
CREATE TRIGGER `after_update_kodeMK`
    AFTER UPDATE ON `matakuliah` FOR EACH ROW
    BEGIN
        SET @tipe = 'UPDATE';
        UPDATE jadwal set Kode_MK = NEW.Kode_MK WHERE Kode_MK = OLD.Kode_MK;
        INSERT INTO tb_log (keterangan, perubahan,waktu) VALUES (NEW.Kode_MK, @tipe,now());
    END;
$$
```

Gambar 7.6. Sintak *Trigger Update* Data Kode Mata Kuliah

Trigger pada sintaks 7.6. akan dieksekusi ketika ada perubahan data pada tabel "matakuliah". Untuk membuktikan hasil eksekusi trigger, ubahlah salah satu kode mata kuliah pada tabel "matakuliah", sebagai contoh kode mata kuliah dengan nilai "1403042111", yaitu mata kuliah "Sistem Operasi" diubah menjadi "1403042117". Kemudian buka tabel "jadwal", apakah kode mata kuliah "1403042111" sudah berubah menjadi "1403042117". *Trigger* pada sintaks 7.5. juga menambahkan data ke dalam tabel "tb_log" ketika terjadi proses *update* data pada tabel "matakuliah". Hasil eksekusi *trigger* ditunjukkan pada gambar 7.7., dimana terdapat satu *record* baru pada tabel "tb_log" dengan nilai pada *field* "perubahan" yaitu "*UPDATE*", dan nilai pada *field* "keterangan" bernilai kode mata kuliah baru, yaitu "1403042117".

log_id	perubahan	waktu	keterangan
36	CREATE	2021-02-08 17:35:34	200304005
37	UPDATE	2021-02-08 17:58:55	1403042117

Gambar 7.7. Hasil Eksekusi *Trigger* pada Proses *Update* Data

3. *Trigger* dengan Pemanggilan *Stored Function*

Selain untuk menjalankan perintah *SELECT*, *INSERT, UPDATE*, atau *DELETE*, perintah di dalam suatu *trigger* dapat juga digunakan untuk memanggil *stored function* untuk mendapatkan nilai kembalian yang dihasilkan oleh *stored function* kemudian menyimpan di dalam variable yang dideklarasikan di dalam *trigger*. Sebagai contoh kasus, *trigger* untuk menghasilkan NIM baru melalui sebuah *stored function* sebelum proses *insert* data mahasiswa baru ke dalam tabel "mahasiswa". Sintaks *trigger* yang melibatkan pemanggilan *stored function* ditunjukkan pada gambar 7.8.

```
DELIMITER $$
CREATE TRIGGER buat_NIM
        BEFORE INSERT ON mahasiswa
        FOR EACH ROW
        BEGIN
                        DECLARE s VARCHAR(12);
                        SET s = (SELECT NIM_Baru_tahun('2020'));
                        SET @tipe = 'CREATE';

                        IF(NEW.Nim IS NULL OR NEW.Nim = '')THEN
                                SET NEW.Nim =s;
                                INSERT INTO tb_log (Nim, perubahan,waktu) VALUES (NEW.Nim, @tipe,now());
                        END IF;
        END;
$$
```

Gambar 7.8. Pemanggilan *Stored Function* pada *Trigger*

Pemanggilan *stored function* melalui *trigger* pada gambar 7.7. dilakukan pada baris sintaks "*SET s = (SELECT NIM_baru());*", nilai kembalian yag dihasilkan oleh *stored function* disimpan ke dalam variable "s". Selanjutnya, ketika terjadi proses penambahan *record* baru pada tabel mahasiswa, nilai *field* Nim akan diisi oleh nilai yang disimpan pada variable "s". untuk menjalankan *trigger* tersebut, eksekusi perintah *query insert* data mahasiswa tanpa mengisikan nilai pada *field* NIM seperti ditunjukkan pada gambar 7.9. Sedangkan hasil eksekusi *trigger* pada gambar 7.7. ditunjukkan pada gambar 7.10.

```
INSERT INTO mahasiswa (Nama_Mhs,Tgl_Lahir,Alamat,Jenis_Kelamin)
VALUES ('Nurdiansyah Suprayoga','1997-08-02','Banteran Sumbang Banyumas','Laki-laki');
```

Gambar 7.9. Mengisi Data Mahasiswa Tanpa NIM

200304004	Anto Kurniawan	1995-08-02	JL. MASJID AL KAUSAR RT. 3 RW. 5 DONDONG KESUGIHAN CILACAP S	Laki-laki
200304005	Wahyu Andreas	1992-08-06	JL STASIUN NOTOG NO 2 RT 06/02 NOTOG PATIKRAJA 53171	Laki-laki
200304006	Nurdiansyah Suprayoga	1997-08-02	Banteran Sumbang Banyumas	Laki-laki

Gambar 7.10. Hasil Eksekusi *Trigger* dengan Pemanggilan *Stored Function*

Berdasarkan gambar 7.10., terlihat pada *field* NIM terisi nilai "200304006". Nilai NIM tersebut dihasilkan oleh *trigger* pada sintaks 7.7., dimana di dalam *trigger* tersebut memanggil *stored function* dengan nama "*NIM_Baru_tahun ( )*" dengan *activation time*-nya adalah *BEFORE INSERT*. Sehingga ketika perintah pada gambar 7.9. dijalankan, sebelum data disimpan ke dalam tabel "mahasiswa", terlebih dahulu akan disimpan NIM bagi data mahasiswa baru tersebut.

C. Rangkuman

1. *Trigger* adalah kumpulan statemen yang disimpan dan dijalankan secara otomatis ketika suatu event terjadi pada suatu kolom atau tabel tertentu.

2. *Trigger* merupakan mekanisme yang baik untuk menjaga integritas data.
3. Pemicu eksekusi statemen di dalam trigger adalah perintah INSERT, UPDATE atau DELETE pada tabel atau kolom tertentu dengan *activation time*-nya adalah *BEFORE* atau *AFTER*.
4. Selain menjalankan perintah DML, di dalam *trigger* dapat juga melakukan pemanggilan *stored function* atau *stored procedure.*

D. Tugas / Latihan

1. Buatlah *stored function* untuk menghasilkan penomoran Nip secara otomatis sesuai berdasarkan Nip terakhir pada tabel "dosen". Selanjutnya buatlah *trigger* untuk menambahkan data dosen baru dengan penomoran Nip merupakan hasil dari *stored function*. Buat juga statemen query pada *trigger* tersebut untuk menambahkan *record* ke dalam tabel "tb_log".
2. Buatlah *trigger* untuk proses *update* kode mata kuliah pada tabel "jadwal" dan "krs" ketika ada perubahan kode mata kuliah pada tabel "mata_kuliah". Buat juga statemen query pada *trigger* tersebut untuk menambahkan *record* ke dalam tabel "tb_log" ketika terjadi perubahan kode mata kuliah pada tabel "mata_kuliah", tabel "jadwal", dan tabel "krs".
3. Buatlah *trigger* untuk proses *update* kode Nip pada tabel "jadwal" ketika tejadi perubahan Nip pada tabel "dosen". Buat juga statemen query pada *trigger* tersebut untuk menambahkan *record* ke dalam tabel "tb_log" ketika terjadi perubahan Nip pada tabel "jadwal", dan tabel "dosen".

E. Daftar Pustaka

1. DuBois, Paul., 2014, *MySQL CookBook 3rd edition,* USA : O'Reilly Media.
2. Harrison, G. & Feuerstein, S., 2006, *MySQL Stored Procedure Programming*, USA : O'Reilly Media.
3. Kruckenberg, M., dan Pipes, J., 2005, Pro MySQL, USA : Apress.
4. Solichin, Achmad, 2010, MySQL 5 : Dari Pemula Hingga Mahir, https://achmatim.net/2010/01/30/buku-gratis-mysql-5-dari-pemula-hingga-mahir/ (diakses tanggal 10 April 2020).